

11

The loop

The working cycle, and the recurring failures of delegated work

The chapters before this one cover what software is; this one covers how the work goes when an assistant writes it: the cycle a task moves through, how big a task should be, when to stop patching and restart, and the failure shapes that belong to the delegation rather than to the code. Those shapes are older than any tool — scope creep, reinvention, drifted conventions, and quiet removals are what delegated work does under a loose brief, whoever the worker — and an assistant produces them at a speed that makes recognizing them worth a chapter.

The model

The cycle

A task moves through seven beats: **specify** what should be true when it’s done; **assemble context** — the files, constraints, and conventions the work needs; **generate**; **verify** at the level the change’s tier calls for (chapter 12); **integrate** — commit it (chapter 10); **clean up** what the change left untidy; **record the decision** if one was made. Most of the beats are familiar from the other chapters; this one is about moving through them deliberately instead of letting “generate” swallow the rest.

DIAGRAM

the seven beats as a ring – specify, assemble context, generate, verify, integrate, clean up, record – with verify pointing out to chapter 12’s tiers and integrate to chapter 10’s commit

Task sizing

The unit of work should equal the unit you can verify. A task is well-sized when you can state its check before starting — “done means a user without access gets zero rows” — and oversized when you can’t, which is the signal to cut it into stages that each have one. Oversized tasks don’t fail loudly: they come back plausible and unverifiable, and get accepted, which is how tier A work ends up at level 4 without anyone deciding that.

The doom loop

A patching session can enter a recognizable state: the same failure returning in new forms, the patch count growing, each fix touching more files than the last. Continuing to patch from inside the accumulated context rarely converges, because the context now contains every wrong assumption that produced the patches. The exit is a restart: a fresh session, a description of the goal rather than of the patches, plus the one or two facts the failed attempts taught you.

Chapter 10 is what makes the restart cheap — discard the branch, stand on the last good commit.

Context hygiene

A session accumulates context, and the context helps until it misleads. Fresh sessions at task boundaries cost a little re-establishment and buy freedom from the previous task’s assumptions; the conventions file below is what does the re-establishing for you. The related habit, from chapter 12: review in a session that didn’t write the code.

Cleanup as scheduled work

A codebase built by generation accumulates a specific kind of debt: two helpers doing the same job, conventions that drift between files written weeks apart, dead code nothing calls, libraries nothing imports any more. Nothing signals any of it — the app works — so cleanup has to be scheduled rather than triggered: a recurring pass with stated asks, the duplication sweep below being one. This is chapter 12’s scheduled-audit rule applied to the codebase itself.

House rules

The instructions file your assistant reads at the start of every session is where working agreements become standing behaviour: the project’s conventions, the risky operations that need asking first, the chapter-specific rules (the authorization block from chapter 6 belongs here). What makes a rule effective is that it requires an action or a statement rather than an attitude — “state which policy protects each new data path” produces a checkable sentence in every summary, where “be careful with security” produces nothing observable. And a rules file is testable: run the same request with and without it, and the difference is what the file buys.

Decisions that stay yours

Some decisions depend on information the assistant doesn’t have: what the product is for and for whom, the shape of your domain, your tolerance for cost and risk, which side of a genuine trade-off to take. These arrive disguised as technical questions, and the common way to delegate one by accident is accepting the first option offered. The recognition habit: when an answer contains a choice among valid alternatives, the choice was yours — notice it, make it, and record it.

A recorded decision — one line on why X over Y — is what stops future sessions, and future you, from relitigating it.

PART

What goes wrong

These five shapes are properties of the diff rather than bugs in the running app, which is why the asks below run on every substantial change rather than on symptoms.

1 Silent scope creep

the change touches what the request never mentioned.

ASK “List everything this change touched that I didn’t ask about, and why. Say nothing else explicitly.”

CHECK the diff’s file list, read against the request.

TELL *files in the diff with no relation to the task.*

2 The second copy

a new implementation of something the project already had.

ASK “Before writing: list existing code in this project that already does part of this. Say none if none.”

CHECK search the project for the older twin of anything newly added.

TELL *a new helper whose name is a synonym of an existing one.*

3 The ignored convention

the change works and matches nothing around it.

ASK “List the conventions the surrounding files follow, and which of them this change doesn’t.”

CHECK the new file next to its oldest neighbour.

TELL *naming, structure, or error handling that differs from every neighbouring file.*

4 The unannounced deletion

the summary describes additions; the diff contains removals.

ASK “List everything this change deletes or disables, with the reason for each. Say nothing if nothing.”

CHECK the diff’s removals, read before the additions.

TELL *minus lines the summary never mentions.*

5 Plausible but wrong

code that reads correctly and does something adjacent to what was asked.

ASK asking helps least here, which is the point of the entry.

CHECK behavioural evidence at the change’s tier — this shape is why chapter 12’s levels exist.

TELL *rarely anything, in the reading — the shape is right and the substance is off by one meaning.*

THE DIRECT TEST

On every substantial diff, ask for three lists before anything else: everything touched that wasn’t asked for, everything deleted, and everything added to the project’s dependencies. Three empty lists are a committable claim — specific enough to be false, and checkable against the diff in a minute. Anything on any of them is a conversation before it’s a commit.

Going deeper

These prompts are for your assistant, and each comes with a note on what a good response looks like.

D1 · Write the house rules

BUILD A TOY

Draft the instructions file for this project. Include: the conventions the codebase already follows (read it and name them), the operations that always need my explicit go-ahead, and rules that require stating something in every summary — which policy protects each new data path, what was deleted, what was touched beyond the request. Then re-run [a recent feature request] in a fresh session with the file in place, and show me the two results side by side.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response earns its rules from your actual codebase rather than generic advice, and the side-by-side is the evidence of what the file buys. Rules that produced no difference are candidates to cut — a rules file also accumulates debt.

D2 · The duplication sweep

AUDIT

List every function or helper in this project that exists in more than one version — same job, different implementations — with the locations of each copy. Then list code that nothing calls, and dependencies nothing imports. Say none explicitly if the project is clean.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response names pairs and their locations, judgeable by opening two files. This is the scheduled cleanup pass; put its date in the calendar, since nothing will ever trigger it.

11

The loop

Going deeper
Where this connects

D3 · The restart rehearsal

Grounded Explanation

Here’s a session where fixing something took many attempts: [paste it, or describe the sequence]. Identify where the patching loop started, what assumption kept every patch from working, and write the specification the restart should have opened with — the goal, plus what the failed attempts established.

What a good response looks like

A good response locates the wrong assumption, and the restart spec it writes is reusable as a template — the goal, the constraint, the learned facts, and nothing about the patches.

Where this connects

Chapter 10 · Version control is the integrate beat and what makes restarts cheap. **Chapter 12 · Verification** is the verify beat, and its tiers are what task sizing protects. **Chapter 6 · Authorization**’s house rules block is this chapter’s rules file, specialized to one surface. **Chapter 9 · Third-party integrations** owns the dependency entry the third list checks for.

D4 · The sizing drill

Prediction Quiz

Quiz me on task sizing. One at a time, propose a feature for my app, and ask me to cut it into tasks where each has a check I can state before starting. Wait for my cut, then critique it: which tasks are still too big to verify, and which checks are vague enough to pass without the work being done.

What a good response looks like

Commit to each cut before the critique. The second half of the critique matters most — a check that can pass vacuously is chapter 12’s proxy green, being designed in.